Reinforcement Learning Lab 2 Report

Georgios Markantonakis 2021030028

Algorithm Implementation and Mathematical Foundations

This section details the theoretical foundations and implementation mechanics of the three multi-armed bandit algorithms utilized: Explore-then-Exploit, Upper Confidence Bound (UCB), and Multiplicative Weights in Bandit mode.

Explore-then-Exploit (EtE)

The Explore-then-Exploit algorithm is a naive strategy that strictly separates the learning phase from the decision-making phase.

Mathematical Formulation:
Given K arms, the algorithm requires a predefined parameter N, representing the number of times each arm must be sampled during the pure exploration phase. For the first $K \times N$ time steps, the algorithm selects arms uniformly:

$$A_t = (t \bmod K) \quad \text{for } t < K \times N \tag{1}$$

During this phase, it computes the empirical mean reward $\hat{\mu}_i$ for each arm i. Once $t \geq K \times N$, the exploration ceases permanently, and the algorithm aggressively exploits the arm with the highest observed mean:

$$A_t = \arg\max_{i \in \{1,\ldots,K\}} \hat{\mu}_i \quad \text{for } t \geq K \times N \tag{2}$$

Upper Confidence Bound (UCB)

The Upper Confidence Bound algorithm resolves the exploration-exploitation dilemma dynamically by applying the principle of optimism in the face of uncertainty.

Mathematical Formulation:
UCB evaluates each arm by combining its empirical mean reward with an exploration term proportional to the variance (uncertainty) of the estimate. The implemented variant calculates the upper bound as:

$$A_t = \arg\max_{i \in \{1,\ldots,K\}} \left(\hat{\mu}_{t,i} + R_{\max} \sqrt{\frac{2 \ln t}{N_t(i)}} \right) \tag{3}$$

where t is the current time step, $N_t(i)$ is the number of times arm i has been selected, and $R_{\max}$ is the theoretical maximum reward used to scale the confidence interval appropriately.

Multiplicative Weights Update (MWU) with Bandit Feedback

The standard Multiplicative Weights algorithm requires full information (knowing the rewards of all actions at every step). To function in a bandit setting (partial feedback), a wrapper class provides custom information to the experts algorithm.

Mathematical Formulation:
The algorithm maintains a set of unnormalized weights $w_{t,i}$ for each arm, initialized to 1. To guarantee a minimum level of exploration, the algorithm mixes the weight-derived distribution

with a uniform distribution, controlled by the parameter γ. The probability of selecting arm i at step t is:

$$q_i^{(t)} = (1-\gamma)\frac{w_{t,i}}{\sum_{j=1}^{K} w_{t,j}} + \frac{\gamma}{K} \tag{4}$$

Because the agent only observes the reward r_t of the chosen arm A_t, it constructs an unbiased estimator of the cost vector using importance sampling. The cost estimate $\hat{c}_i^{(t)}$ is defined as:

$$\hat{c}_i^{(t)} = \begin{cases} \frac{-r_t}{q_i^{(t)}} & \text{if } i = A_t \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

The weights are then updated using the learning rate parameter h and a scaling factor $S = K/\gamma$. The implemented multiplicative update rule is:

$$w_{t+1,i} = w_{t,i} \times (1-\eta)^{\frac{\hat{c}_i^{(t)}}{S}} \tag{6}$$

If we set $\gamma = \eta = \sqrt[3]{\frac{n \ln n}{T}}$, then the algorithm has $O(3(n \ln n)^{1/3} T^{2/3})$ regret.

Part A: Reverse-Engineering Non-Stationarity

Sanity check

By conducting a sanity check on the algorithms, their implementation can be tested, to see if they actually perform as expected:

Arm	$\mathbf{p_L}$	$\mathbf{p_H}$	$\mathbf{T_L}$	$\mathbf{T_H}$	**Initial State**
Arm 1	0.4	0.8	100	100	L
Arm 2	0.4	0.8	100	100	L

Table 1: Parameter Configuration for the Sanity Check Environment

Using the above parameters, we get:

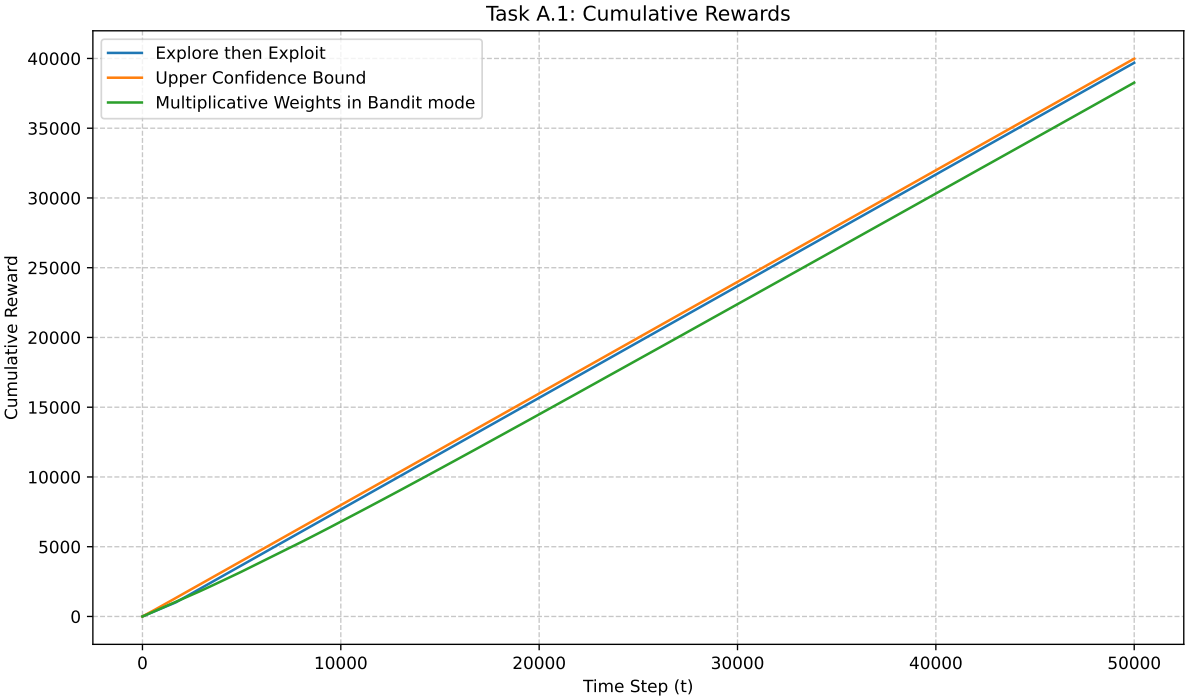

Figure 1: Cumulative reward comparison.

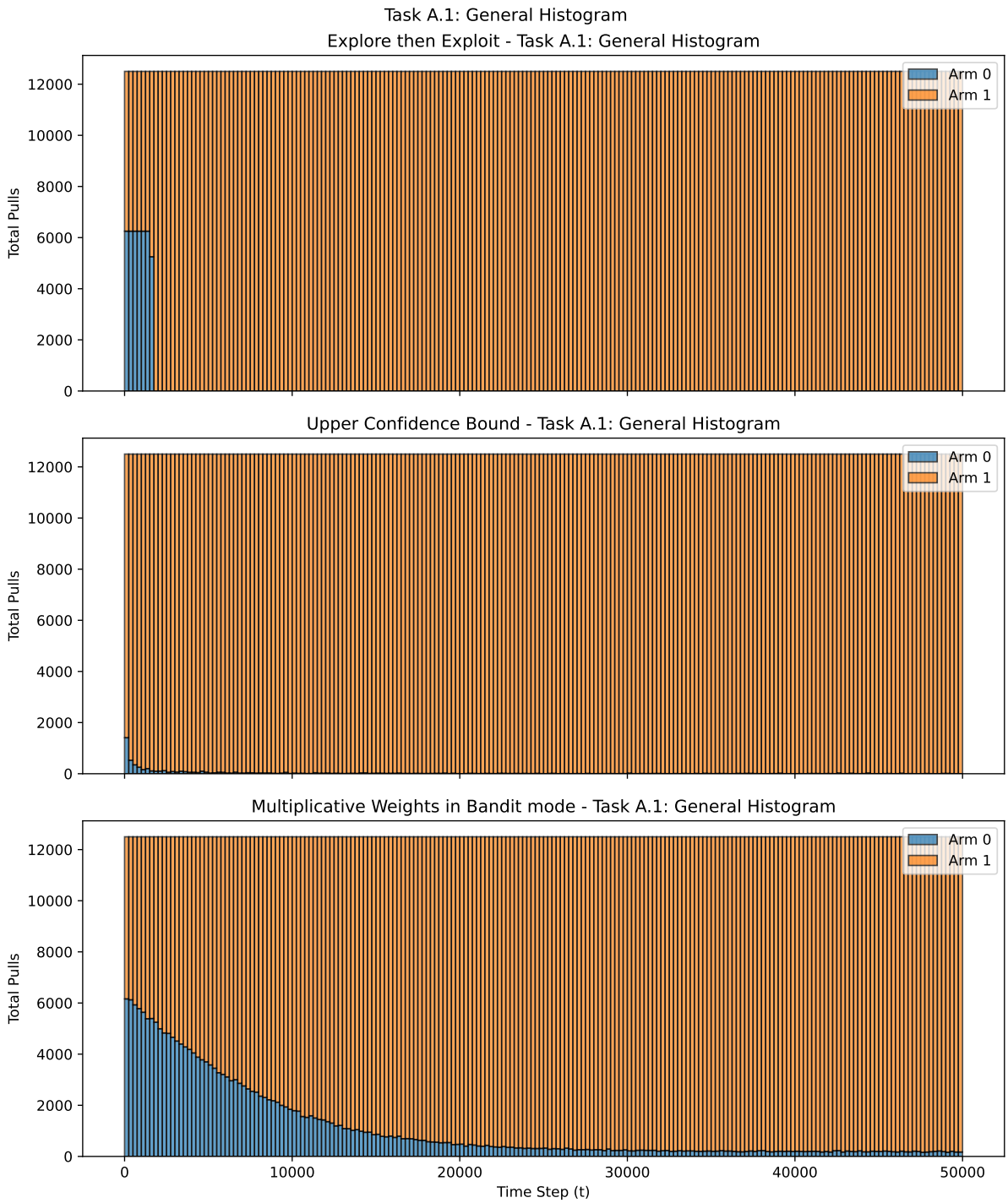

Figure 2: Arm selection.

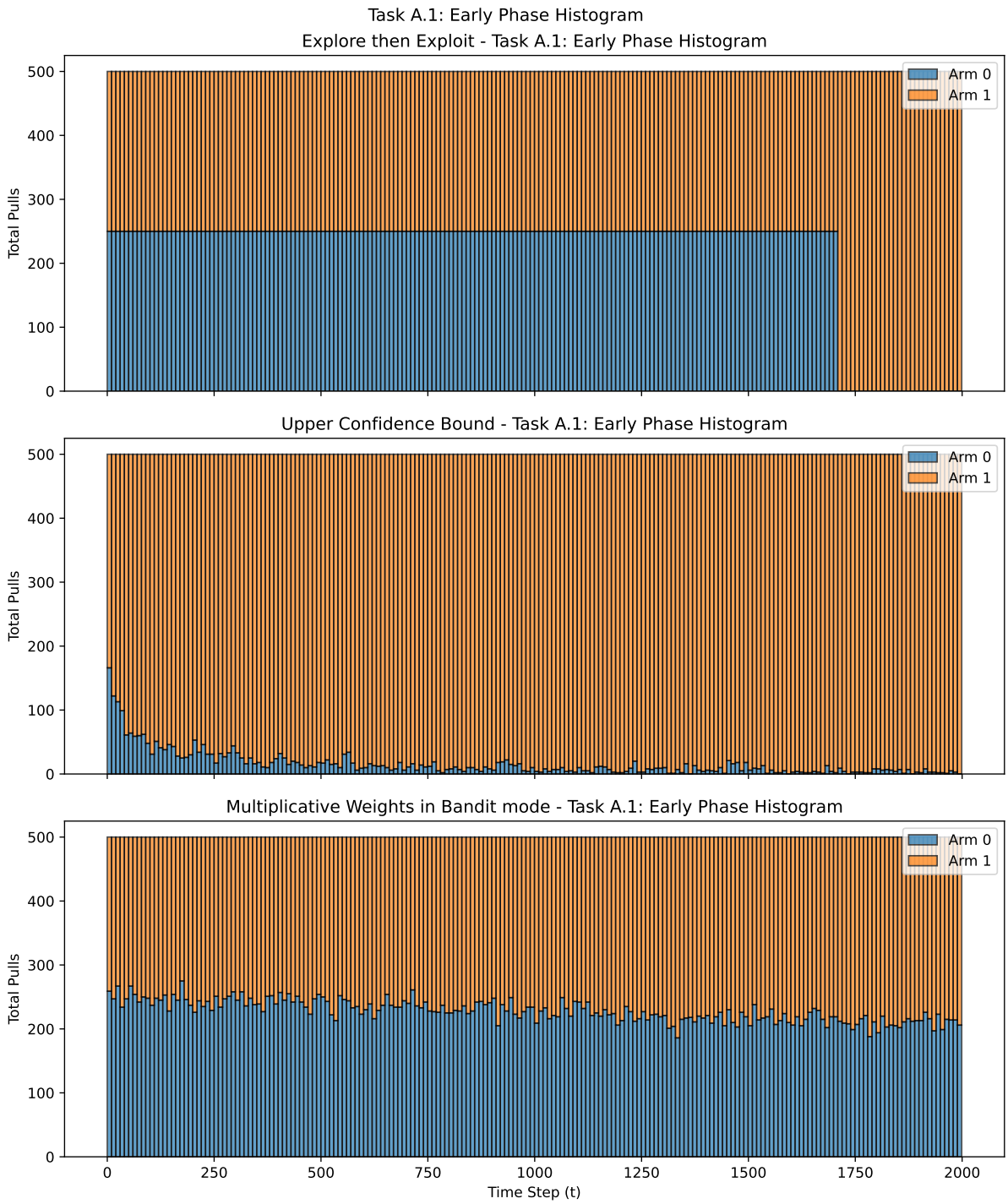

Figure 3: Early stage Arm selection.

The environment works as expected. EtE should outrank all algorithms, since it always chooses the best option after exploration, while algorithms choose different arms to test if they have changed.

0.1 Sanity Check Evaluation: Algorithmic Adaptability to Non-Stationarity

To evaluate the algorithms' resilience to non-stationary dynamics, a specific "sanity check" environment was constructed. The environment spans $T = 50,000$ time steps and utilizes $K = 2$ arms. Let $E \approx 2(T/2)^{2/3}$ represent the total duration of the Explore-then-Exploit

exploration phase.

Arm	$\mathbf{p_L}$	$\mathbf{p_H}$	$\mathbf{T_L}$	$\mathbf{T_H}$	**Initial State**
Arm 1 (Index 0)	0.1	0.9	$T - E$	E	H
Arm 2 (Index 1)	0.1	0.9	E	$T - E$	L

Table 2: Parameter Configuration for the Sanity Check Environment

EtE vs MW on non-stationary environment

The simulation results conclusively demonstrate that the Multiplicative Weights Update (MWU) algorithm drastically outperforms the Explore-then-Exploit (EtE) strategy.

The environment is explicitly engineered as a "bait-and-switch" trap for stationary algorithms:

- **The Bait (Phase 1):** During the first E time steps, EtE conducts its pure exploration phase, pulling both arms uniformly. During this exact window, Arm 1 is in its high-yield state ($p_H = 0.9$) and Arm 2 is in its low-yield state ($p_L = 0.1$). EtE correctly concludes that Arm 1 is the optimal choice.
- **The Switch (Phase 2):** Immediately after EtE concludes its exploration and permanently commits to exploiting Arm 1, the environment's internal periodic timers trigger a phase shift. Arm 1 plummets to a 10% success rate, while Arm 2 rises to 90%.

EtE lacks a mechanism to return to exploration. Once it commits to Arm 1 at step E, its learning rate drops to zero. It spends the remaining $T - E$ steps (the vast majority of the simulation) blindly pulling a suboptimal arm, accumulating massive linear regret. MWU avoids this trap because it never stops learning. The continuous exploration parameter γ ensures that MWU occasionally samples Arm 2 even while Arm 1 appears optimal. When the environment phase-shifts, MWU's cost estimates instantly capture the dropping reward rate of Arm 1 and the rising reward rate of Arm 2. The exponential update rule seamlessly shifts the algorithm's probability mass, allowing MWU to adaptively transition its exploitation to Arm 2 and secure the high-yield rewards for the remainder of the horizon.

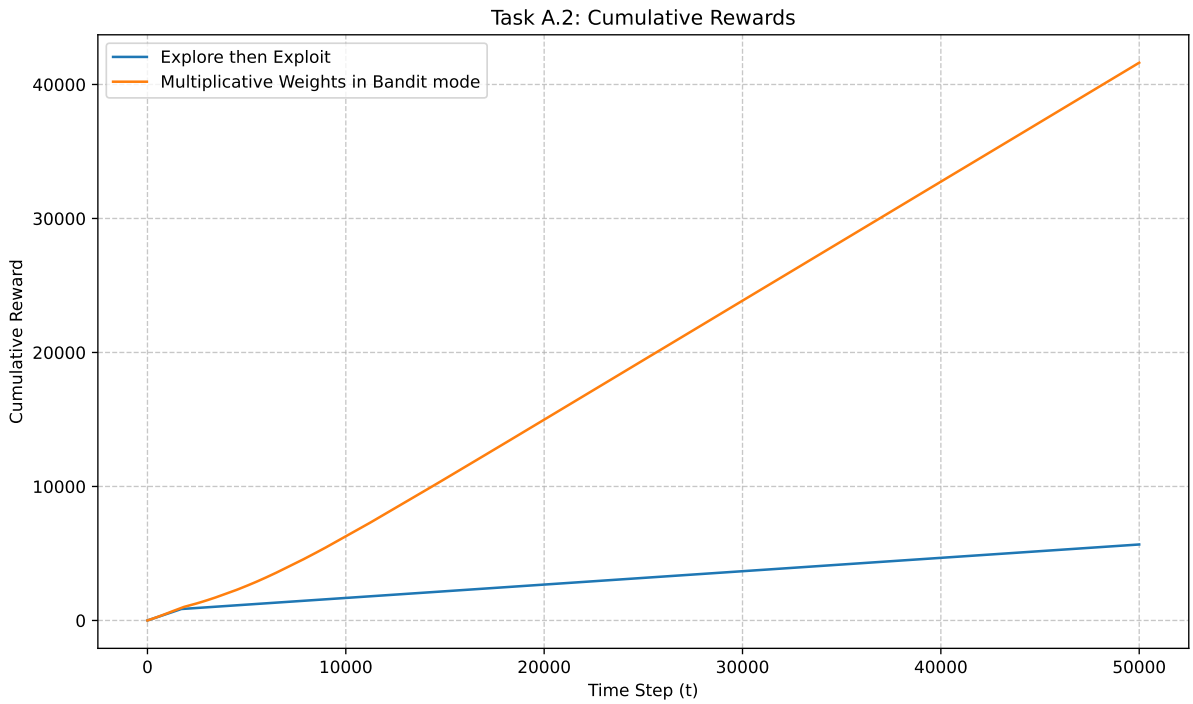

Figure 4: Cumulative reward comparison.

Figure 5: Arm selection.

UCB vs MW on non-stationary environment

Arm	p_L	p_H	T_L	T_H	Initial State
Arm 1	0.1	0.9	1	1	L
Arm 2	0.1	0.9	1	1	H

Table 3: Parameter Configuration for the Non-Stationary Phase-Shift Environment

The UCB algorithm fails on this environment, because it always chooses the arm with the lowest probability. If the environment transitioned slowly, then the UCB algorithm will often choose the arm with the highest reward, while MW will still try and choose both, to check if the environment changed. Also, the initial state should be like this, since UCB implementation always starts at arm 1.

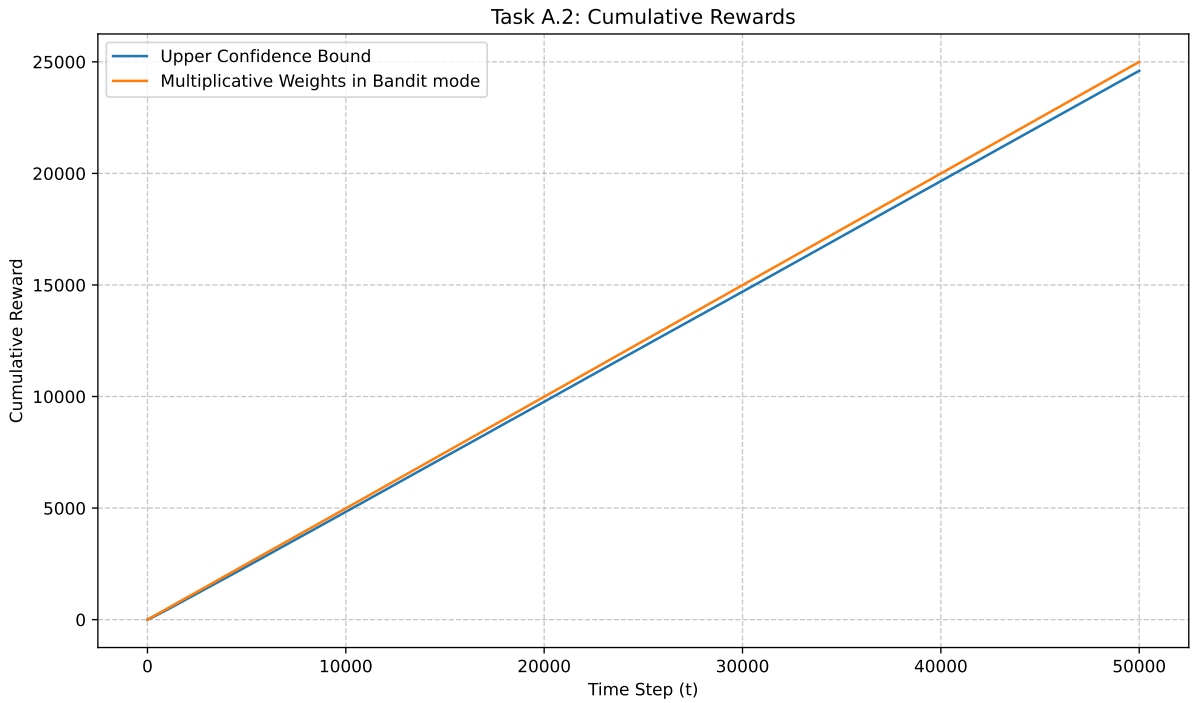

Figure 6: Cumulative reward comparison.

Figure 7: Arm selection.

Part B: Price Wars

Environment Definition

The second phase of the experiment transitions the learner price war against adversarial competitor. The learner selects from a set of $K = 10$ discrete prices, uniformly initialized in the interval $[0, 5]$.

Customer purchasing decisions are modeled using a choice function. Let p_t be the learner's chosen price and q_t be the competitor's chosen price. The probability that the customer buys from the learner is defined as:

$$\mathbb{P}(\text{buy from learner}) = \frac{1}{1 + \exp(\beta(p_t - q_t))} \tag{7}$$

The parameter β dictates the price sensitivity of the market. A low β implies brand loyalty or indifference to price disparities, whereas a high β represents a highly elastic, hyper-competitive market where the customer strictly prefers the cheaper option. Realized rewards are the revenue generated from a successful sale.

Adversarial Mechanisms

Unlike the stochastic setting, the competitor in this regime calculates a mathematically perfect best response to maximize its own expected revenue, given complete knowledge of the learner's strategy.

- **Against a Deterministic Learner (UCB):** Because UCB plays a pure strategy, the competitor observes the exact chosen price p_t before finalizing its own decision. It searches a highly granular continuous space to find the optimal undercutting price:

$$q^{(t)} = \arg\max_{q} \left[q \cdot \left(1 - \frac{1}{1 + \exp(\beta(p_t - q_t))} \right) \right] \tag{8}$$

- **Against a Randomized Learner (MWU):** Because MWU plays a mixed strategy, the competitor cannot observe the specific price before it is pulled. Instead, it observes the learner's full probability distribution $\boldsymbol{p}^{(t)}$. The competitor calculates the expected reward across the learner's entire discrete price grid and parks at the globally optimal continuous price:

$$q^{(t)} = \arg\max_{q} \sum_{k=1}^{K} p_k^{(t)} \left[q \cdot \left(1 - \frac{1}{1 + \exp(\beta(p_k - q))} \right) \right] \tag{9}$$

Results and Analysis

The simulation was executed over $T = 5000$ steps for 50 Monte Carlo epochs across three market sensitivity regimes: $\beta = 0.5$ (Low), $\beta = 2.0$ (Medium), and $\beta = 10.0$ (High).

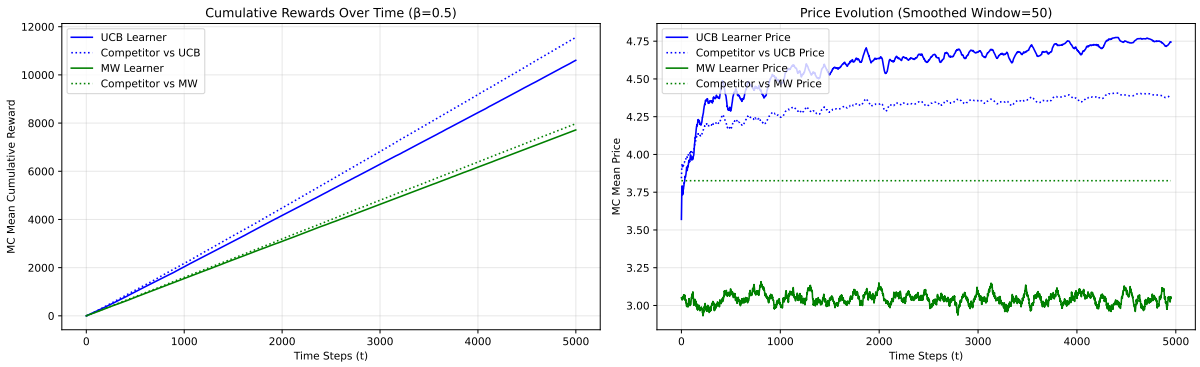

Figure 8: Cumulative Rewards and Price Evolution in a Low-Sensitivity Market ($\beta = 0.5$).

Low Sensitivity Dynamics: In the $\beta = 0.5$ regime, the market is forgiving. UCB thrives because the competitor's undercutting does not guarantee a lost sale. UCB can confidently select high-margin prices, yielding superior cumulative rewards. MWU underperforms relative to UCB because its defensive randomization acts as an unnecessary exploration tax in a non-punishing environment.

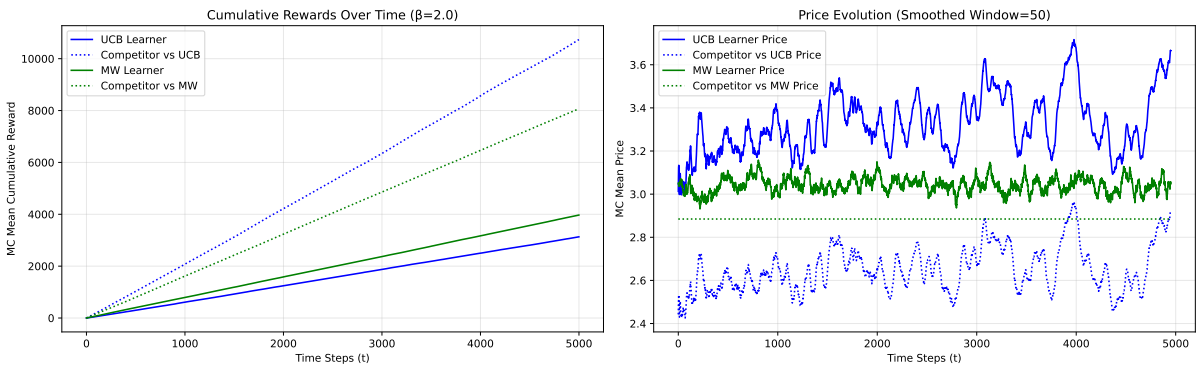

Figure 9: Cumulative Rewards and Price Evolution in a Medium-Sensitivity Market ($\beta = 2.0$).

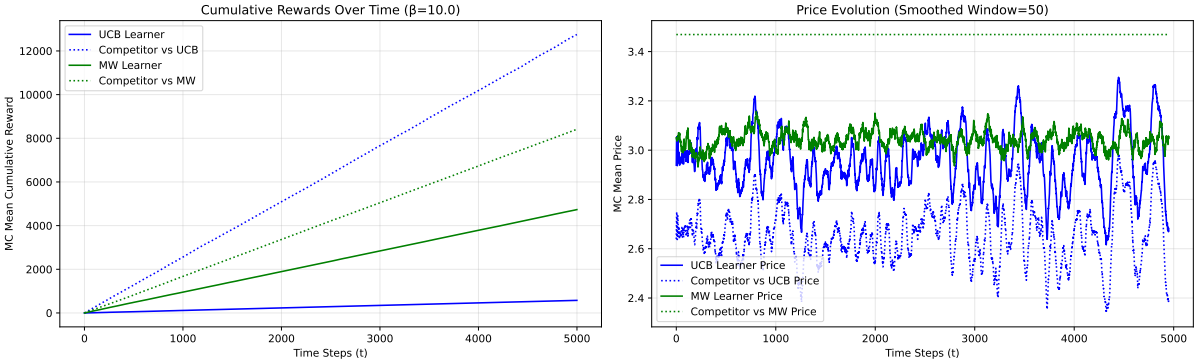

Figure 10: Cumulative Rewards and Price Evolution in a High-Sensitivity Market ($\beta = 10.0$).

High Sensitivity Dynamics: The $\beta = 10.0$ regime perfectly illustrates the fatal vulnerability of deterministic algorithms. Because the market strictly punishes higher prices, the adversary's perfect prediction of UCB's pure strategy allows it to continuously undercut the learner by a fraction of a cent. UCB falls into an unrecoverable cyclic feedback loop, yielding a near-zero cumulative reward.

Conversely, MWU demonstrates profound adversarial robustness. By maintaining a probability distribution, MWU prevents the competitor from aggressively undercutting. The competitor is mathematically forced to adopt a static, conservative baseline price (evidenced by the flat dotted line), guaranteeing the MWU learner a baseline survival income and sub-linear regret even against an omniscient adversary.